Amar Telidji University Laghouat

Department of Computer Science

Level: 4th Year Engineering

Course: Network Security

Lab 04: Firewall Exploration

Supervised by:

Mr. Nasredine LAGRAA

Prepared by:

Smail MERSAD

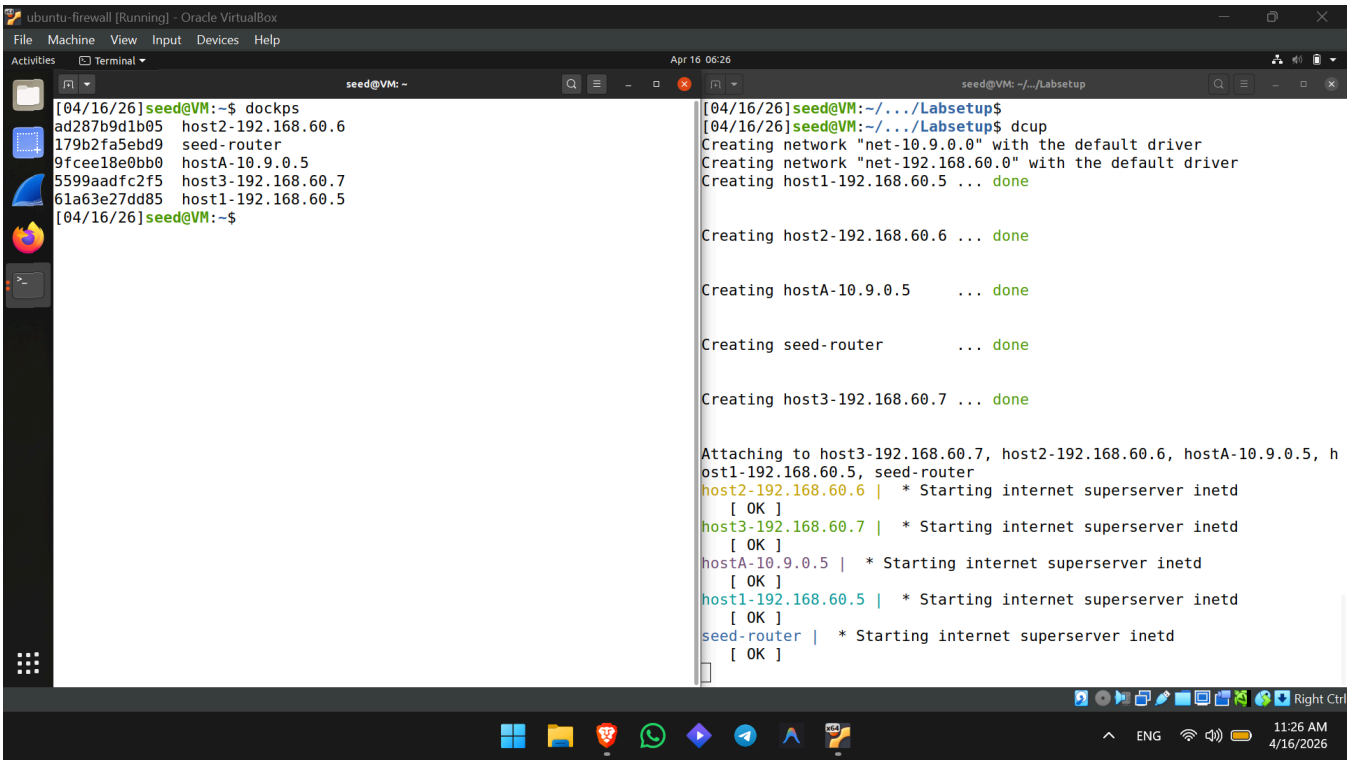
Academic Year: 2025 / 2026

1. Lab Environment Setup

The goal of this lab is to understand how firewalls work by implementing a simple packet-filtering firewall and using iptables . The lab environment consists of an attacker machine, a router, and an internal network with several hosts.

Execution Steps:

1. We navigated to the Labsetup directory on the VM.
2. We built the container images using the provided docker-compose.yml file via the alias $ dcbuild.
3. We started the containers in the background using $ dcup.
4. We verified the running containers using $ dockps.

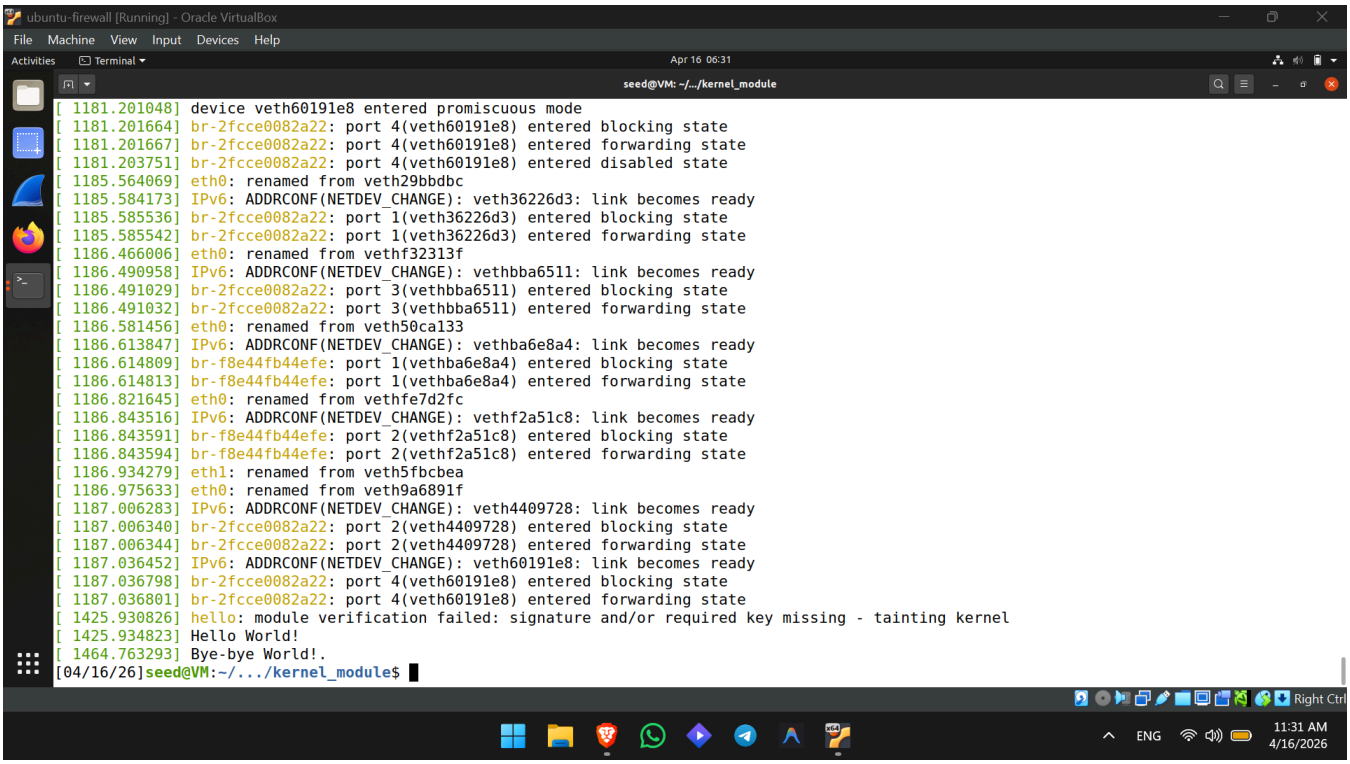


2. Task 1.A: Implement a Simple Kernel Module

Objective: To become familiar with Loadable Kernel Modules (LKM), which allow us to add new functionalities (like packet filtering) to the Linux kernel at runtime without needing to rebuild the kernel or reboot the computer .

Execution Steps:

1. On the host VM, we located the provided hello.c file and its corresponding Makefile.
2. We compiled the code into a loadable kernel module by running the $ make command, which generated the hello.ko file.
3. We loaded the module into the kernel using $ sudo insmod hello.ko.
4. We verified it was loaded by listing the active modules with $ lsmod | grep hello.
5. We removed the module using $ sudo rmmod hello.
6. Finally, we checked the kernel logs using $ dmesg to view the output generated by our module's initialization and cleanup functions .



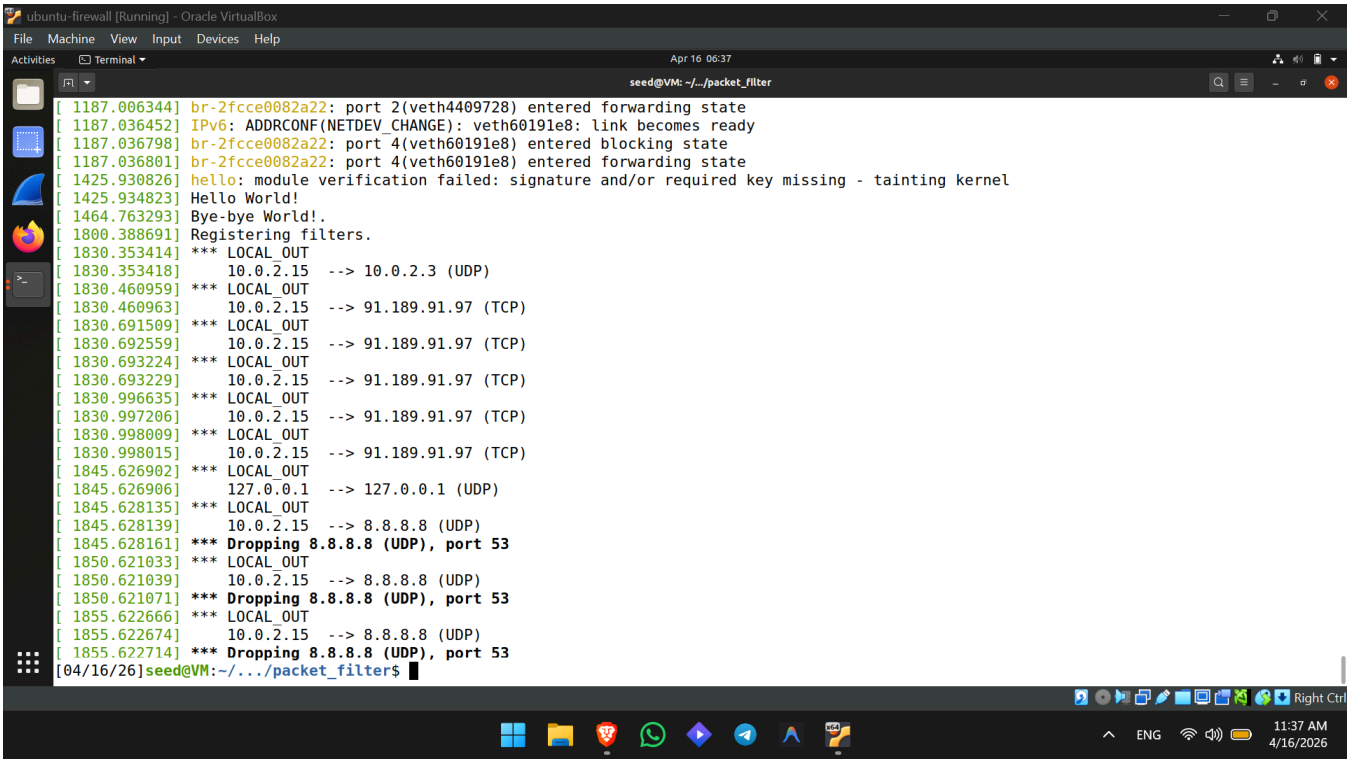
Observation: By observing the kernel logs, we confirmed that our LKM executed successfully. When the module was inserted using `insmod`, the initialization function was triggered, printing "Hello World!" to the `/var/log/syslog` file . When the module was removed using `rmmod`, the cleanup function was triggered, printing "Bye-bye World!". This confirms we can dynamically inject code into the kernel, which is the foundational step for injecting custom firewall rules into the packet processing path.

3. Task 1.B: Implement a Simple Firewall Using Netfilter

Objective: To implement a packet filtering module by hooking functions to Netfilter inside the Linux kernel.

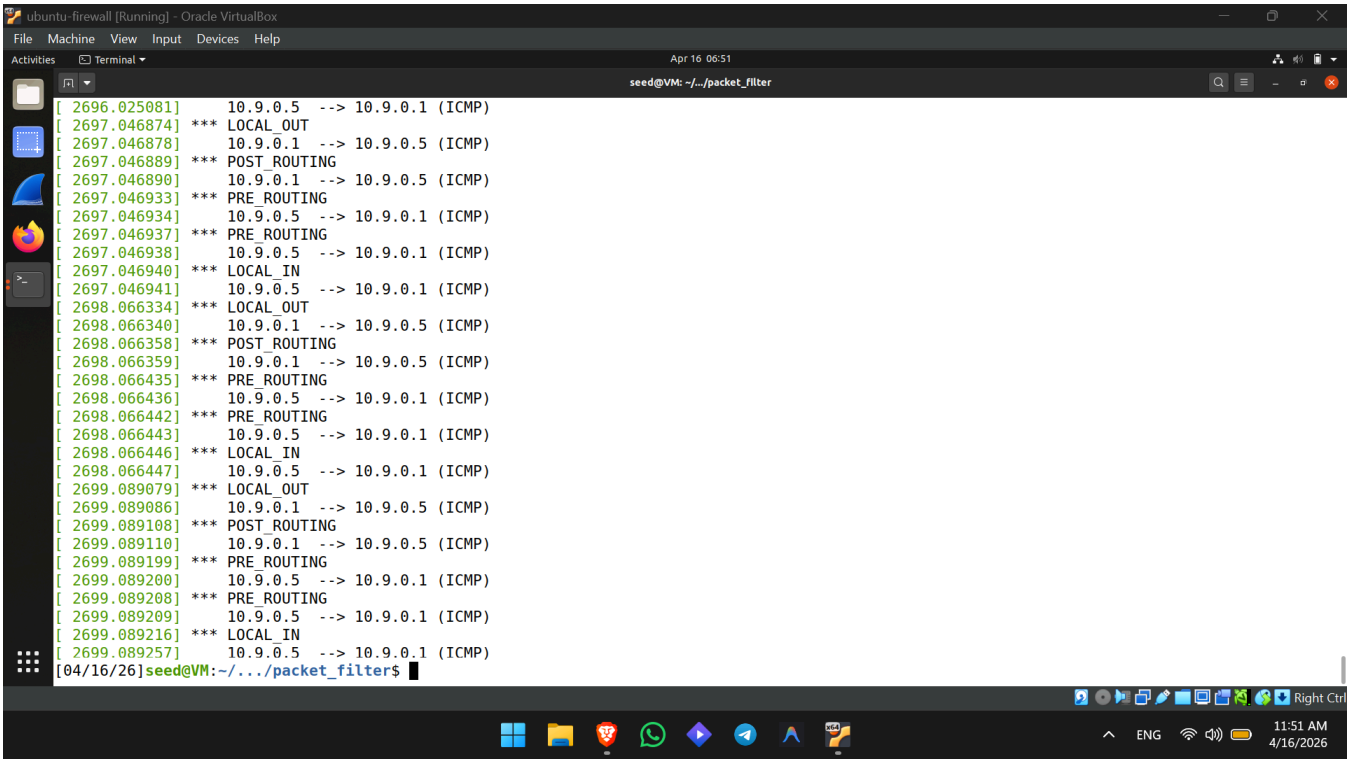
Sub-Task 1: Testing the Sample Code

- **Execution & Observation:** We compiled and loaded the `seedFilter.ko` module. We then executed the command `$ dig @8.8.8.8 www.example.com` to query Google's DNS . The command timed out, and upon checking `dmesg`, we observed the kernel module successfully intercepted and dropped the UDP packet destined for port 53 on 8.8.8.8 .



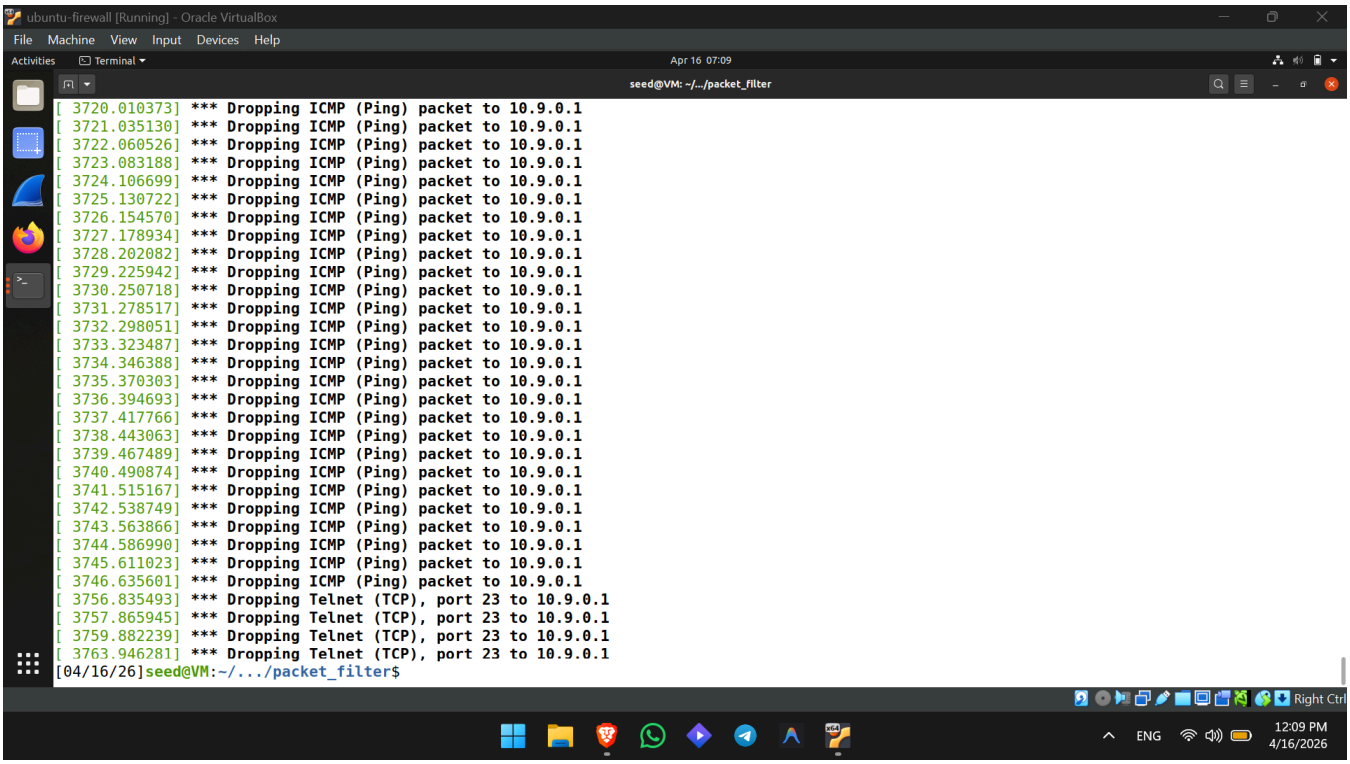
Sub-Task 2: Hooking `printInfo` to All Netfilter Hooks

- **Code Modification:** We modified the `seedFilter.c` file to register `printInfo` across all five available Netfilter hooks.
- **Observation & Explanation:** By generating network traffic and analyzing the `dmesg` output, we observed the packet lifecycle through the Netfilter architecture:
    - `NF_INET_PRE_ROUTING`: Invoked for all incoming packets before any routing decisions are made.
    - `NF_INET_LOCAL_IN`: Invoked for incoming packets explicitly destined for the local machine after routing.
    - `NF_INET_FORWARD`: Invoked for incoming packets that are destined for another host (the machine is acting as a router).
    - `NF_INET_LOCAL_OUT`: Invoked for locally generated packets before routing.
    - `NF_INET_POST_ROUTING`: Invoked for all outgoing packets right before they leave the network interface.



Sub-Task 3: Blocking Ping and Telnet

- **Code:** *[Insert the C code snippets for your ICMP blocking function and TCP blocking function here]*
- **Execution & Observation:** We implemented two custom hook functions to block ICMP echo requests and TCP connections targeting port 23. We registered both to the `NF_INET_LOCAL_IN` hook because these packets are destined for the host machine itself (10.9.0.1) . To test, we accessed the `10.9.0.5` container and executed `$ ping 10.9.0.1` and `$ telnet 10.9.0.1` . Both requests hung indefinitely and failed. Checking the host's `dmesg` confirmed our custom Netfilter hooks actively intercepted and dropped the respective packets, demonstrating a successfully implemented stateless firewall within the kernel.



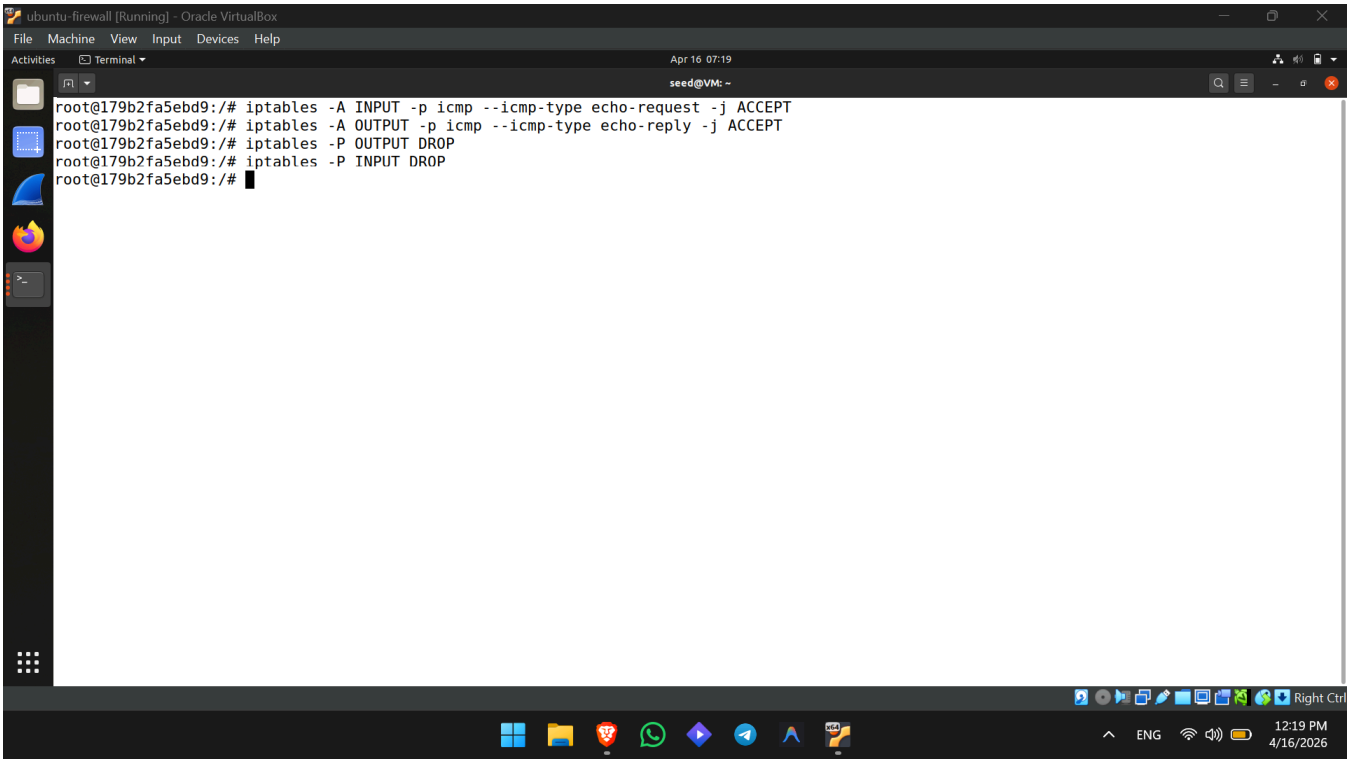
4. Task 2.A: Protecting the Router

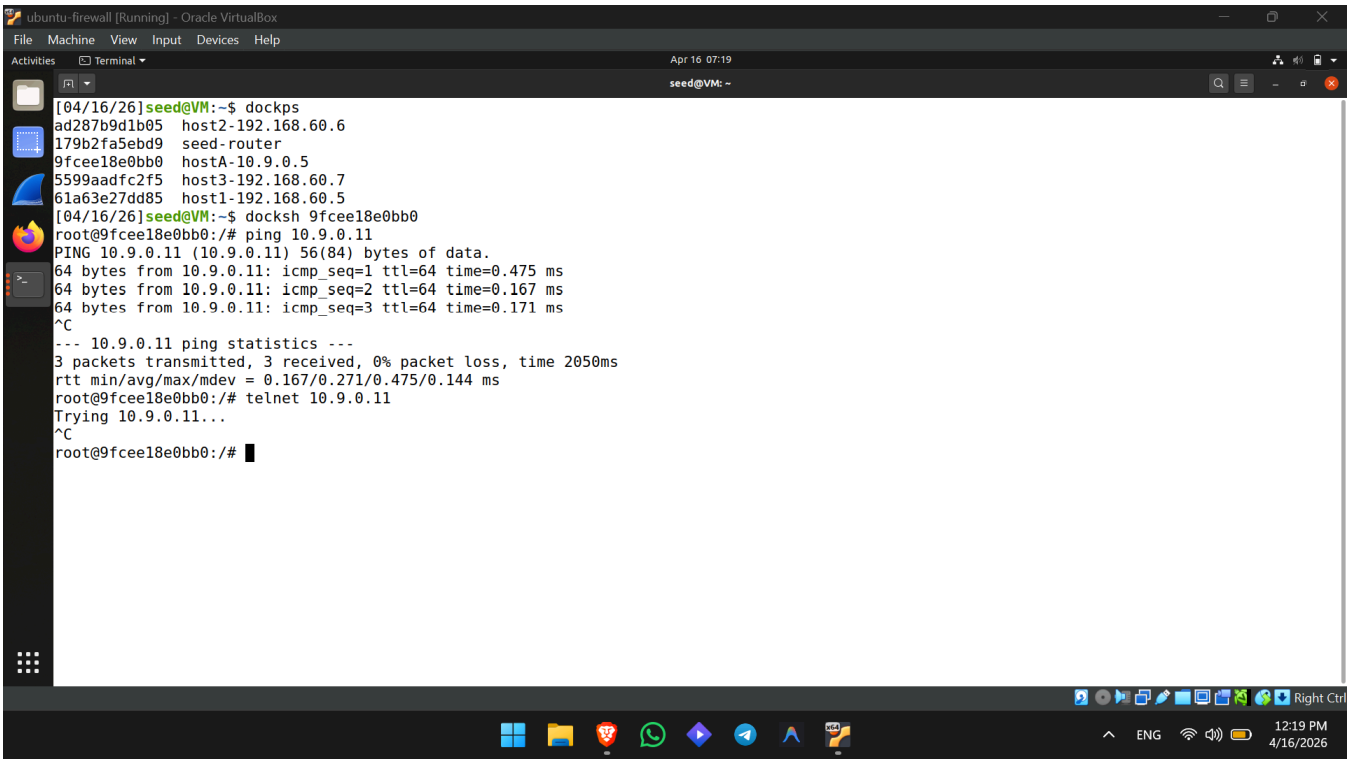
Objective: To configure iptables on the Router machine to block all incoming connections except for ICMP echo requests (ping) .

Commands and Explanations:

We executed the following rules on the Router container:

- iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT: This appends (-A) a rule to the INPUT chain. It targets the ICMP protocol (-p icmp) specifically looking for ping requests (--icmp-type echo-request) and tells the firewall to accept them (-j ACCEPT).
- iptables -A OUTPUT -p icmp --icmp-type echo-reply -j ACCEPT: This appends a rule to the OUTPUT chain to allow the router to send ICMP ping replies back to the requester.
- iptables -P OUTPUT DROP: This changes the default policy (-P) of the OUTPUT chain to DROP. If an outgoing packet doesn't match any specific rule above, it will be dropped .
- iptables -P INPUT DROP: This changes the default policy of the INPUT chain to DROP. Any incoming packet that isn't explicitly accepted by a rule is dropped.





Observations: We tested the firewall from the 10.9.0.5 host.

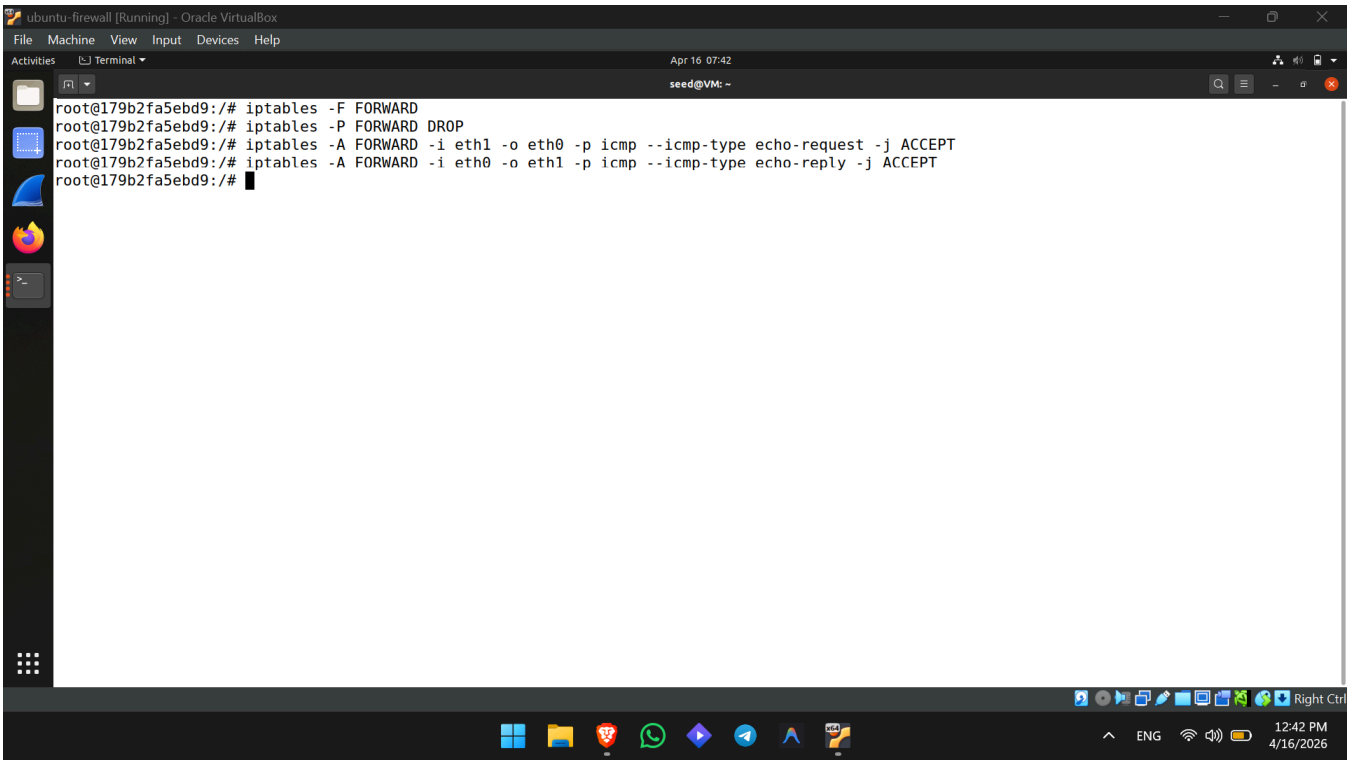
1. **Can you ping the router?** Yes. The ping succeeded because our first two explicit rules caught the ICMP echo requests and replies, allowing them through before the default DROP policy could act on them .
2. **Can you telnet into the router?** No. The telnet connection timed out. Because we did not add a specific rule to accept TCP packets on port 23, the telnet packets fell through to the default INPUT DROP policy and were discarded by the firewall .

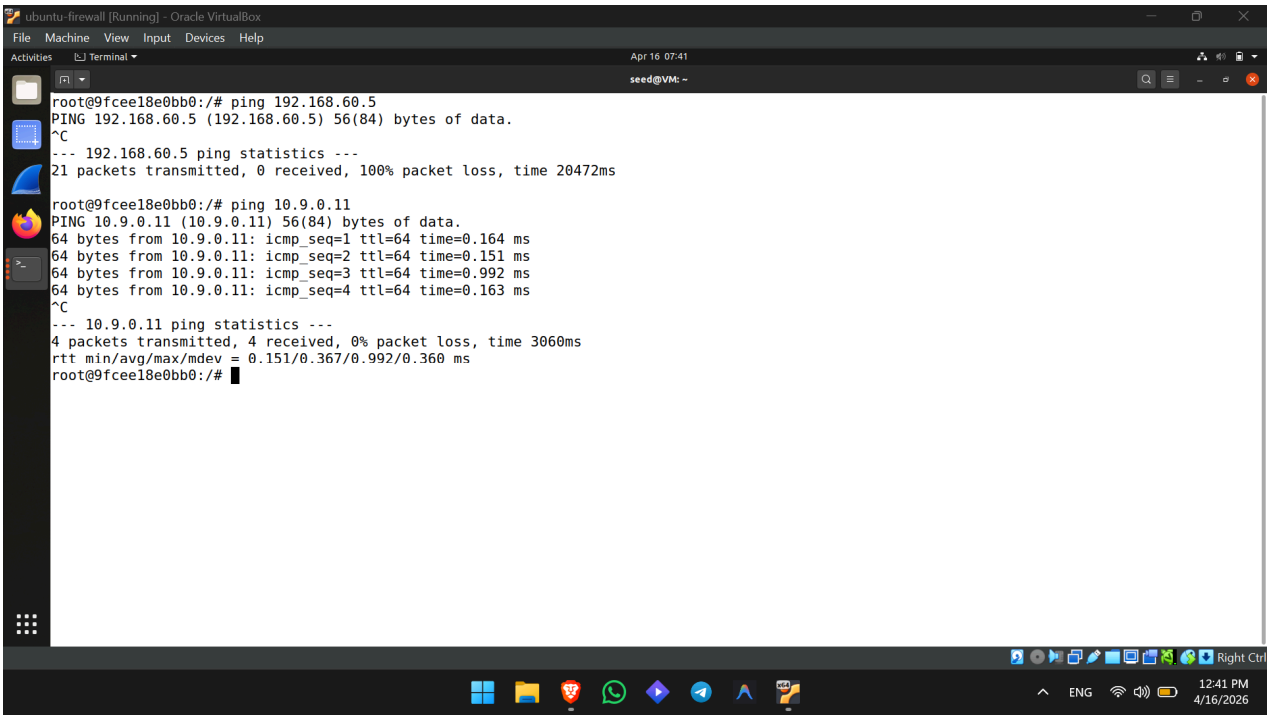
5. Task 2.B: Protecting the Internal Network

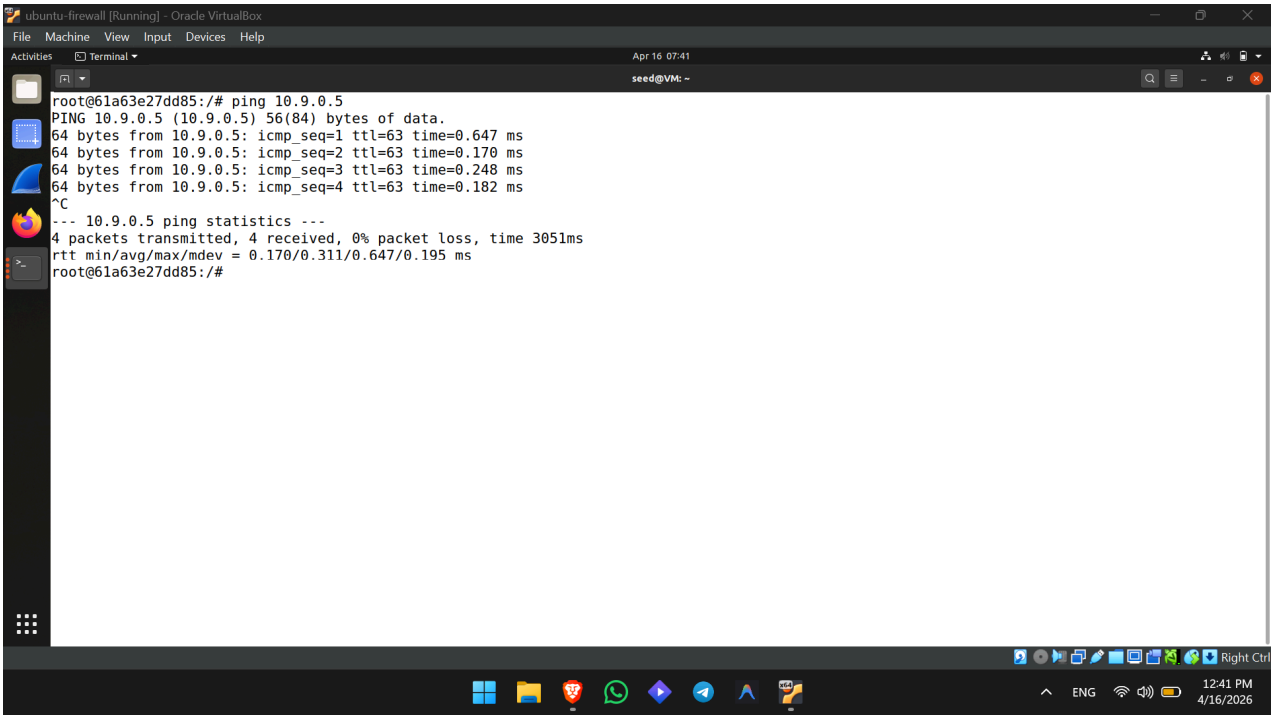
Objective: To implement firewall rules on the Router using the FORWARD chain to protect the 192.168.60.0/24 internal network while allowing specific outgoing ICMP traffic .

Commands and Explanations: First, we identified our network interfaces using ip addr: eth0 for the external network and eth1 for the internal network . We then applied the following rules on the Router:

1. iptables -P FORWARD DROP: This sets the default policy for the FORWARD chain to drop all packets passing through the router. This explicitly satisfies Objective 4 (blocking all other packets between networks). By defaulting to DROP, this also satisfies Objective 1, as external pings to internal hosts will be blocked without an explicit allow rule.
2. iptables -A FORWARD -i eth1 -o eth0 -p icmp --icmp-type echo-request -j ACCEPT: This rule permits ICMP echo requests originating from the internal interface (eth1) and heading to the external interface (eth0) .
3. iptables -A FORWARD -i eth0 -o eth1 -p icmp --icmp-type echo-reply -j ACCEPT: This rule allows the corresponding ICMP echo replies to return from the external network back into the internal network. Together with the previous rule, this satisfies Objective 3.







Observations: We validated our firewall using ping tests from both sides of the network:

- **Outside hosts cannot ping internal hosts:** A ping from `10.9.0.5` to `192.168.60.5` failed because the `FORWARD` default policy dropped the `echo-request` packets.
- **Outside hosts can ping the router:** A ping from `10.9.0.5` to `10.9.0.11` succeeded. This is because packets destined *for* the router traverse the `INPUT` chain, not the `FORWARD` chain, and our `INPUT` policy was reset to `ACCEPT`.
- **Internal hosts can ping outside hosts:** A ping from `192.168.60.5` to `10.9.0.5` succeeded because our explicit rules allowed the `echo-request` out and the `echo-reply` back in.